

Parte I. Descripción General

1.1. Descripción del proyecto

El presente proyecto consiste en el diseño, desarrollo e implementación de una **Plataforma Web de Estudio Colaborativo**. Esta herramienta nace de la necesidad de centralizar en un único entorno digital la gestión de grupos de trabajo académico, el intercambio de recursos didácticos y la comunicación fluida entre estudiantes. A diferencia de las redes sociales genéricas o los LMS (*Learning Management Systems*) institucionales, esta solución se enfoca en la interacción horizontal "entre pares" (*peer-to-peer learning*), facilitando la autoorganización de los alumnos.

La plataforma permite a los usuarios registrarse y crear o unirse a "Salas de Estudio" categorizadas por asignaturas o temas de interés. Dentro de estos espacios, los integrantes disponen de herramientas para compartir apuntes y bibliografía de forma estructurada, así como canales de comunicación síncrona (chat en tiempo real) y asíncrona (repositorios de archivos). El sistema está diseñado bajo una arquitectura de **Single Page Application (SPA)**, garantizando una experiencia de usuario ágil y dinámica similar a una aplicación de escritorio.

Técnicamente, el proyecto aborda el reto de la interactividad en tiempo real y la gestión eficiente de contenidos multimedia. Se implementará un sistema de roles para asegurar la moderación de los grupos y se integrarán notificaciones instantáneas para mantener a los usuarios actualizados sobre la actividad de sus compañeros, fomentando así el compromiso y la continuidad en el estudio.

1.2. Objetivo general y justificación / Objetivos

Objetivo General Desarrollar una aplicación web escalable que fomente el aprendizaje colaborativo, proporcionando a los estudiantes un entorno virtual dedicado donde puedan organizar grupos de estudio, compartir conocimiento y comunicarse en tiempo real, mejorando así su rendimiento académico y reduciendo el aislamiento en el estudio remoto.

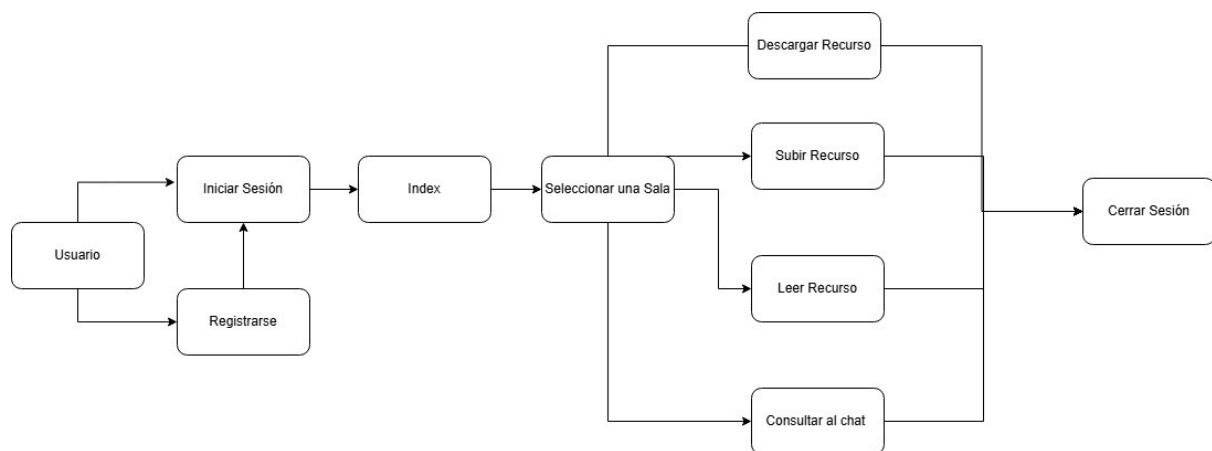
Justificación: La **motivación** principal de este proyecto surge del auge de la educación híbrida y a distancia. Las herramientas actuales suelen estar fragmentadas: los alumnos usan WhatsApp para hablar, Google Drive para archivos y Zoom para reuniones, lo que dispersa la información. Este proyecto se justifica por la necesidad de **unificar estas funcionalidades** en un ecosistema diseñado pedagógicamente para el estudio, optimizando el tiempo del alumno y facilitando el acceso a recursos compartidos.

Objetivos Específicos

1. **Implementar un sistema de autenticación seguro:** Garantizar la privacidad y seguridad de los datos de los usuarios mediante el uso de tokens (JWT) y cifrado de contraseñas, permitiendo la gestión de perfiles personalizados.

2. **Desarrollar un módulo de gestión de recursos:** Crear una estructura de base de datos eficiente (NoSQL) que permita almacenar, categorizar y recuperar archivos y enlaces compartidos dentro de cada grupo de estudio.
3. **Integrar comunicación en tiempo real:** Implementar tecnología de WebSockets para habilitar funcionalidades de chat instantáneo y notificaciones push, elementos críticos para la coordinación de grupos.
4. **Diseñar una interfaz intuitiva y responsiva:** Utilizar librerías de componentes modernos para asegurar que la plataforma sea accesible desde distintos dispositivos, priorizando la usabilidad (UX/UI).
5. **Excluir dispositivos móviles:** Usualmente a la hora de estudiar los móviles son una distracción y por lo general los estudiantes intentan evitarlos.

1.3. Esquema general de funcionamiento



1.4. Funcionalidades principales / Alcance

El alcance del proyecto se estructura en tres módulos funcionales principales, cubriendo las necesidades básicas de la gestión colaborativa:

1. Gestión de Usuarios y Accesos:

- Registro e inicio de sesión seguro (Login/Logout).
- Edición de perfil de usuario (biografía, estudios).
- Recuperación de credenciales básica.
- Sistema de roles (Administrador de la plataforma vs. Usuario estándar).

2. Gestión de Grupos y Recursos (Core):

- Creación, edición y eliminación de grupos de estudio (CRUD).
- Búsqueda y filtrado de grupos por asignatura o nombre.
- Subida y descarga de archivos (PDFs, imágenes, resúmenes).
- Compartir enlaces externos de interés.
- Organización de recursos por carpetas o etiquetas dentro del grupo.

3. Comunicación y Tiempo Real:

- Chat grupal integrado en cada sala de estudio.
- Notificaciones en tiempo real (ej. "Nuevo mensaje en tu grupo", "Nuevo archivo subido").
- Historial de mensajes persistente.
- Programación de eventos/sesiones (calendario sencillo de próximas reuniones).

Exclusiones (Fuera del alcance):

- No se implementará un sistema de videoconferencia nativo (se permitirá compartir enlaces a Zoom/Teams, pero el vídeo no se procesará en la plataforma).
- No se desarrollará una aplicación móvil nativa (Android/iOS), aunque la web será *responsive* (adaptable a móviles).
- No se incluye pasarela de pagos ni funciones "Premium".

1.5. Análisis de la competencia

En el mercado existen herramientas consolidadas, aunque pocas combinan todas las funcionalidades específicas para estudiantes en una sola interfaz gratuita:

- **Discord:**
 - *Descripción:* Plataforma de chat y voz muy popular entre jóvenes.
 - *Comparativa:* Aunque es excelente para la comunicación en tiempo real (ventaja), su gestión de archivos y "hilos" académicos es caótica y poco estructurada para el estudio serio (desventaja frente a nuestro proyecto).
- **Moodle / Blackboard:**
 - *Descripción:* Plataformas institucionales usadas por universidades.
 - *Comparativa:* Son muy rígidas y centradas en la comunicación Profesor-Alumno. Nuestro proyecto fomenta la comunicación Alumno-Alumno, siendo más ágil y menos burocrática.
- **Notion:**
 - *Descripción:* Herramienta de productividad y notas.
 - *Comparativa:* Permite compartir recursos de forma excelente, pero carece de chat en tiempo real integrado y funciones sociales dinámicas (descubrimiento de grupos).

1.6. Análisis DAFO

Análisis de la viabilidad del proyecto mediante la matriz de Debilidades, Amenazas, Fortalezas y Oportunidades:

- **Debilidades (Internas):**
 - *Dependencia de la masa crítica:* Una plataforma social no es útil si no hay suficientes usuarios activos y contenido compartido al inicio.
 - *Limitación de almacenamiento:* El alojamiento de archivos pesados puede requerir recursos de servidor costosos si el proyecto escala mucho.
- **Amenazas (Externas):**
 - *Competencia desleal:* Grandes empresas (Microsoft, Google) podrían integrar funcionalidades similares en sus suites educativas gratuitas.

- *Seguridad*: Riesgo de subida de archivos maliciosos o contenido con derechos de autor (copyright) por parte de los usuarios.
- **Fortalezas (Internas):**
 - *Tecnología de vanguardia*: El uso de una SPA (Single Page Application) y WebSockets ofrece una experiencia de usuario superior a los foros universitarios tradicionales.
 - *Nicho específico*: Al estar diseñada *por y para* estudiantes, la UX (Experiencia de Usuario) está adaptada a flujos de trabajo académicos reales, evitando distracciones.
- **Oportunidades (Externas):**
 - *Digitalización educativa*: La tendencia post-pandemia hacia modelos híbridos hace que estas herramientas sean más necesarias que nunca.
 - *Escalabilidad*: Posibilidad futura de añadir "gamificación" (puntos por ayudar) o monetización mediante tutorías privadas.

1.7. Marco tecnológico

Para la implementación se ha seleccionado un stack tecnológico basado en JavaScript/TypeScript, lo que permite un desarrollo unificado y eficiente:

- **Frontend (Interfaz):**
 - **React**: Librería de JavaScript para construir interfaces de usuario dinámicas y modulares.
 - **Tailwind CSS**: Framework de estilos "utility-first" que permite un diseño rápido, consistente y totalmente adaptativo (responsive).
- **Backend (Servidor):**
 - **Node.js con Express**: Entorno de ejecución y framework ligero para crear la API REST. Destaca por su manejo eficiente de múltiples conexiones simultáneas (I/O no bloqueante).
 - **[Socket.io](#) o WebSocket**: Librerías para gestionar la comunicación bidireccional en tiempo real (Chat y notificaciones).
 - **Laravel**: Para gestionar el servidor web del proyecto.
- **Base de Datos:**
 - **MongoDB**: Base de datos NoSQL orientada a documentos. Se elige por su flexibilidad para guardar datos heterogéneos (perfiles, mensajes de chat, metadatos de archivos) sin un esquema rígido inicial.
- **Herramientas de Desarrollo:**
 - **Git/GitHub**: Para el control de versiones y trabajo colaborativo del equipo.
 - **Postman**: Para el testeo de los endpoints de la API, junto con Laravel.

Parte 2 Análisis y Diseño técnico

2.1 Requisitos

2.1.1 Requisitos funcionales

Módulo de Gestión de Usuarios y Seguridad

- **Registro de Usuarios:** El sistema debe permitir el registro de nuevos estudiantes solicitando correo electrónico, nombre de usuario y contraseña.
- **Autenticación:** El sistema debe permitir el inicio de sesión (Login) validando las credenciales contra la base de datos y generando un token de sesión (JWT) para el acceso seguro.
- **Gestión de Perfil:** El usuario podrá editar su perfil personal, añadiendo información biográfica, estudios en curso y una imagen de avatar.

Módulo de Gestión de Grupos (Salas de Estudio)

- **Creación de Salas:** El usuario podrá crear nuevas "Salas de Estudio", definiendo un nombre, una descripción y la asignatura o tema de interés asociado.
- **Búsqueda y Filtrado:** El sistema debe ofrecer un buscador que permita localizar grupos existentes por nombre, asignatura o etiquetas.
- **Unirse/Salir de Grupos:** El usuario podrá unirse a salas de estudio existentes para acceder a sus contenidos y abandonar las mismas cuando lo desee.
- **Moderación de Grupo:** El creador del grupo (o moderador asignado) podrá eliminar a participantes que incumplan las normas de la sala.

Módulo de Gestión de Recursos (Repositorio)

- **Carga de Archivos:** El sistema debe permitir a los usuarios subir archivos (formatos PDF e imágenes) al repositorio de cada grupo de estudio.
- **Descarga de Recursos:** Los miembros del grupo podrán visualizar y descargar los materiales compartidos por otros pares.
- **Enlaces Externos:** El sistema debe permitir compartir y previsualizar enlaces a recursos externos (webs, vídeos de YouTube, artículos).
- **Organización de Contenido:** El sistema permitirá etiquetar o categorizar los recursos subidos para facilitar su recuperación posterior.

Módulo de Comunicación y Tiempo Real

- **Chat Grupal:** El sistema debe proporcionar un chat de texto en tiempo real dentro de cada sala de estudio, visible para todos los miembros activos de dicha sala..
- **Calendario de Eventos:** El sistema permitirá a los usuarios programar eventos sencillos (fechas de reuniones o exámenes) visibles en un calendario o lista dentro del grupo.

2.1.2 Requisitos no funcionales.

Rendimiento y Eficiencia

- **Reproductor de video:** los enlaces de video serán redirigidos a paginas externas para que la web funcione mejor
- **Arquitectura SPA:** La aplicación debe comportarse como una Single Page Application (SPA), evitando la recarga completa de la página al navegar entre secciones internas para garantizar fluidez.

Seguridad y Privacidad

- **Cifrado de contraseñas:** Las contraseñas de los usuarios no deben almacenarse en texto plano; deben ser encriptadas (hashing) antes de guardarse en la base de datos.
- **Seguridad de Sesión:** La gestión de sesiones debe realizarse mediante tokens seguros (JWT), los cuales tendrán un tiempo de expiración definido.

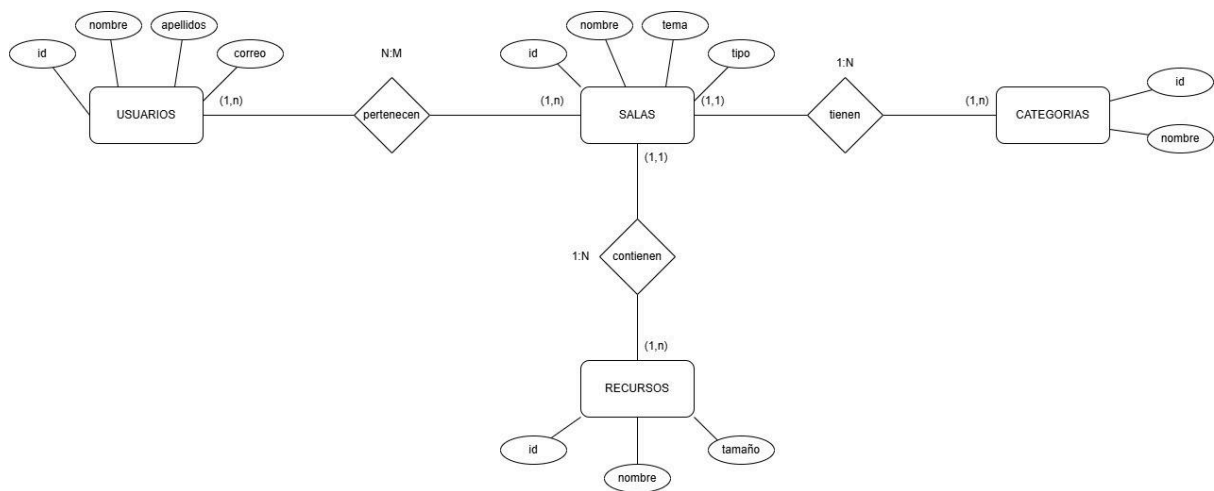
Usabilidad e Interfaz

- **Enfoque Desktop:** La interfaz de usuario (UI) estará optimizada prioritariamente para resoluciones de escritorio y portátiles (pantallas grandes), fomentando el estudio inmersivo y minimizando distracciones propias de entornos móviles.
- **Diseño Responsivo:** Aunque el enfoque es de escritorio, la interfaz debe ser responsiva y adaptarse visualmente si el usuario redimensiona la ventana del navegador, sin romper la maquetación.

Restricciones Tecnológicas y de Implementación

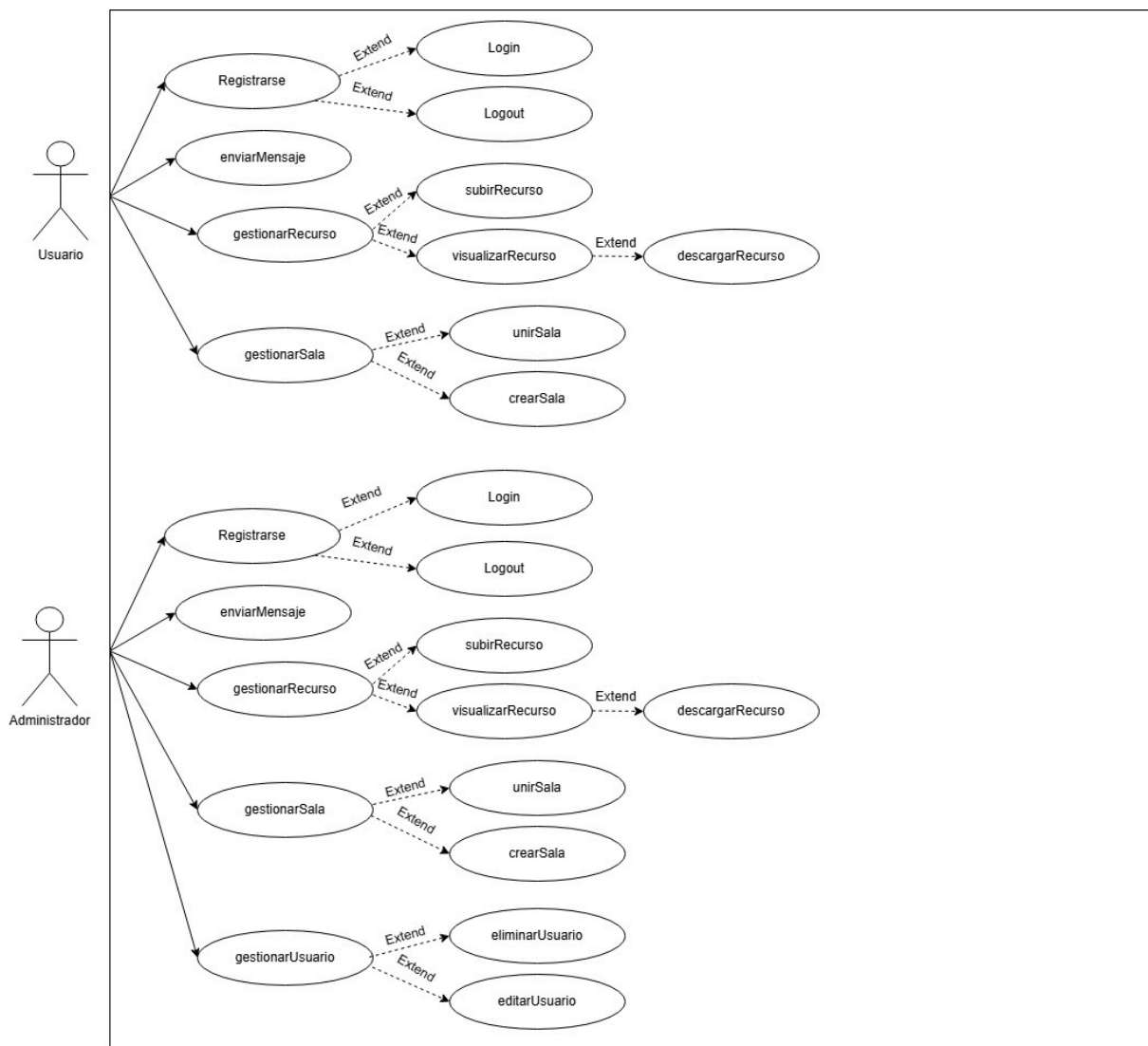
- **Stack Tecnológico:** El sistema debe desarrollarse utilizando el stack MERN (MongoDB, Express, React, Node.js) y Socket.io para la comunicación en tiempo real.
- **Almacenamiento de Datos:** Se utilizará una base de datos NoSQL (MongoDB) para permitir flexibilidad en la estructura de los datos de los grupos y mensajes.
- **Exclusión de Vídeo Nativo:** El sistema no procesará streaming de vídeo ni videoconferencias de forma nativa; estas funcionalidades se delegarán a servicios externos mediante enlaces (Zoom, Teams, etc.).

2.1.3. Diseño físico/lógico de datos



2.1.4 Diseño funcional

2.1.4.1 Casos de Uso



Nombre: Login
Descripción: El administrador o usuario solicita hacer el login después de estar registrado
Actor: Administrador y Usuario.
Precondiciones: Se requiere que esté ya registrado.
Curso normal del caso de uso: 1. El administrador o usuario solicita el login 2. El sistema busca si está registrado o no 3. El sistema acepta el login 4. El administrador o usuario tienen acceso
Postcondiciones: El sistema muestra la siguiente pantalla después de registrarse
Alternativas: 5. El sistema no encuentra los datos utilizados ya que no está registrado

Nombre: Logout
Descripción: El administrador o usuario solicita hacer el login después de haber navegado por la web
Actor: Administrador y Usuario.
Precondiciones: Se requiere que esté ya haya navegado por la web.
Curso normal del caso de uso: 1. El administrador o usuario solicita el logout 2. El sistema acepta el logout. 3. El administrador o usuario ya no tienen acceso
Postcondiciones: El sistema muestra la página de inicio de sesión otra vez
Alternativas: 4. Si hay inactividad se hará el logout automático.

Nombre: subirRecurso
Descripción: El administrador o usuario suben un recurso para poder utilizarlo
Actor: Administrador o Usuario.
Precondiciones: Se requiere que haya iniciado sesión y este en alguna sala
Curso normal del caso de uso: 1. El administrador o usuario selecciona el archivo que quiere subir. 2. El sistema acepta el archivo y lo sube en la sala . 3. El sistema muestra el archivo subido listo para utilizar
Postcondiciones: El usuario o administrador podrán editarlo, descargarlo y guardarse el archivo
Alternativas: 4. El archivo pesa mucho y no deja subirlo o está en un formato que no está permitido

Nombre: descargarRecurso
Descripción: El administrador o usuario descargan un recurso para poder utilizarlo
Actor: Administrador o Usuario.
Precondiciones: Se requiere que haya iniciado sesión y este en alguna sala
Curso normal del caso de uso: 1. El administrador o usuario selecciona el archivo que quiere descargar. 2. El sistema filtra el archivo y lo descarga..
Postcondiciones: Si el archivo tiene cambios se informará para volverlo a descargar
Alternativas: 3. Abrir el archivo sin necesidad de descargarlo

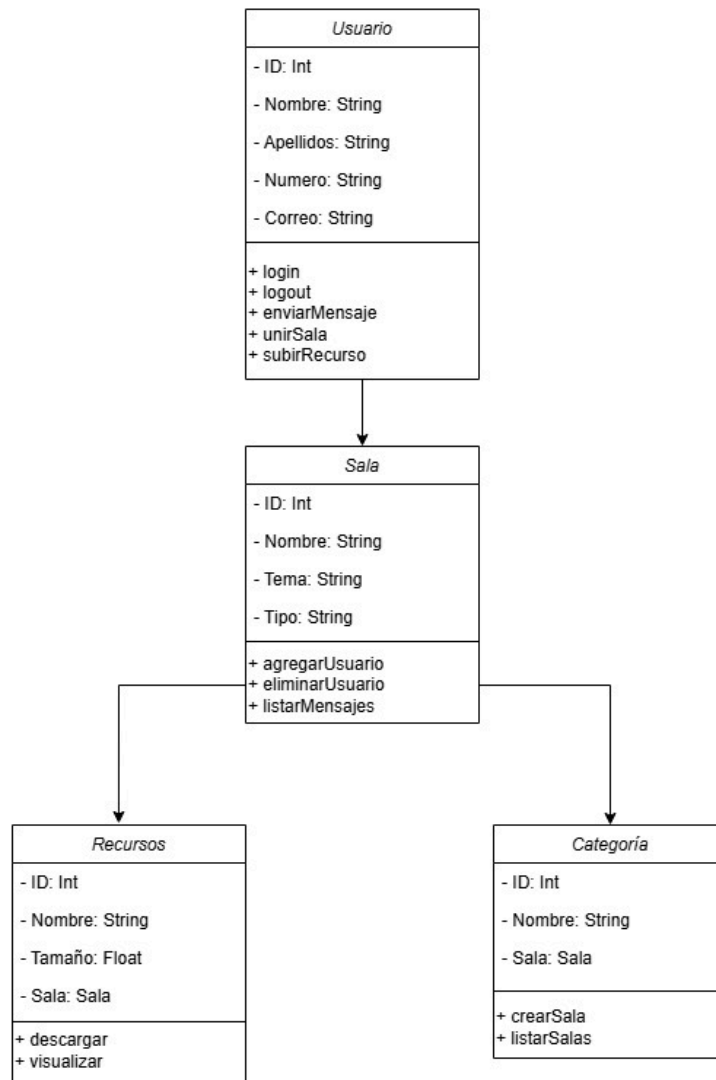
Nombre: unirSala
Descripción: El administrador o usuario se pueden unir a cualquier sala
Actor: Administrador o Usuario.
Precondiciones: Haber iniciado sesión
Curso normal del caso de uso: 1. El administrador o usuario inicia sesión 2. El administrador o usuario solicita acceso a la sala 3. El sistema acepta la solicitud y los añade a la sala 4. El administrador o usuario estarán dentro de la sala y tendrán libertad
Postcondiciones: Abandonar la sala al finalizar.
Alternativas: 5. Podrá crear la sala directamente antes de buscar.

Nombre: crearSala
Descripción: El administrador o usuario pueden crear una sala para estar con otros usuarios
Actor: Administrador o Usuario.
Precondiciones: Se requiere haber iniciado sesión
Curso normal del caso de uso: 1. El administrador o usuario inicia sesión 2. El administrador o usuario crean la sala con ciertas características 3. El sistema acepta la creación de la sala 4. El administrador o usuario tendrán la sala creada lista para trabajar
Postcondiciones: Abandonar la sala al finalizar
Alternativas: 5. Podrá buscar antes de crear la sala a ver si hay alguna otra sala que te interese.

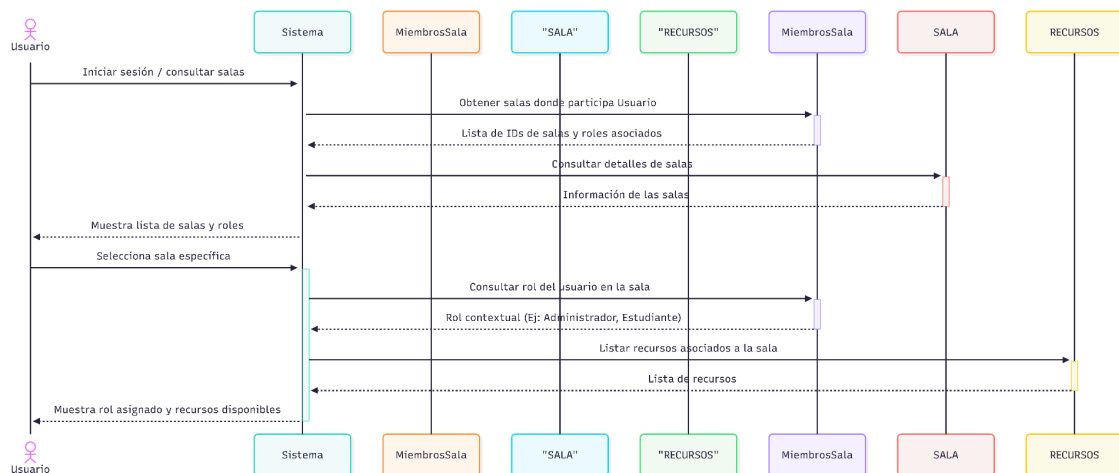
Nombre: eliminarUsuario
Descripción: El administrador podrá eliminar cualquier usuario
Actor: Administrador.
Precondiciones: Se requiere que sea el propio administrador.
Curso normal del caso de uso: 1. El administrador selecciona al usuario. 2. El administrador selecciona eliminar usuario. 3. El sistema busca el usuario que el administrador quiera eliminar. 4. El sistema elimina ese usuario.
Postcondiciones: El usuario eliminado no podrá volver a usar su cuenta para acceder
Alternativas: 5. El administrador en vez de eliminar al usuario le puede dar un toque de atención.

Nombre: editarUsuario
Descripción: El administrador podrá editar cualquier usuario
Actor: Administrador.
Precondiciones: Se requiere que sea el propio administrador.
Curso normal del caso de uso: 1. El administrador selecciona al usuario. 2. El administrador selecciona editar usuario. 3. El sistema busca el usuario que el administrador quiera editar. 4. El administrador edita algún atributo de ese usuario. 5. El sistema guarda los cambios.
Postcondiciones: El usuario sabrá que se le ha editado su propio perfil.
Alternativas: 6. El administrador avisa al usuario para que lo edite el mismo antes de que lo haga el propio administrador.

2.1.4.2 Diagrama de Clases



2.1.4.3 Diagrama de Secuencia



Parte 3 Planificación y memoria económica

Para el desarrollo de la Plataforma Web de Estudio Colaborativo, la planificación se ha estructurado en fases secuenciales y ágiles, estimando una duración total de **12 semanas**. A continuación, se presenta una tabla representativa del cronograma (diagrama de Gantt lógico) y la valoración económica.

Planificación de Tareas (Estimación temporal)

Fase	Tareas Principales	Duración	Recursos Necesarios
1. Análisis y Diseño	Toma de requisitos, diseño de BBDD (MongoDB), diseño de interfaces (Figma) y arquitectura.	Semanas 1 - 2	Analista/Diseñador, Figma, draw.io.
2. Desarrollo Backend	Configuración Node.js/Express, API REST, autenticación JWT y configuración MongoDB.	Semanas 3 - 5	Desarrollador Backend, Postman, Git.

3. Desarrollo Frontend	Maquetación con React y Tailwind CSS, integración de SPA, rutas privadas.	Semanas 6 - 8	Desarrollador Frontend, React Router.
4. Integración Tiempo Real	Implementación de WebSockets (Socket.io) para chat y notificaciones.	Semanas 9 - 10	Equipo Full-Stack.
5. Pruebas y Despliegue	Testing unitario e integración, corrección de bugs, despliegue en servidor.	Semanas 11 - 12	QA Tester, Servicios Cloud (ej. AWS/Heroku).

Memoria Económica

La siguiente tabla refleja los costes estimados de personal y recursos de infraestructura para la fase de desarrollo:

Concepto / Rol	Horas Estimadas	Coste/Hora (€)	Coste Total (€)
Jefe de Proyecto / Analista	60	35	2.100
Desarrollador Frontend	120	25	3.000
Desarrollador Backend	120	25	3.000

Infraestructura y Software (Cloud, dominios)	N/A	N/A	300
TOTAL ESTIMADO	300 horas		8.400 €

Análisis de monetización y mantenimiento: El coste de mantenimiento mensual (servidores, base de datos en la nube) se estima bajo en la fase inicial. Dado que el proyecto no incluye funciones "Premium" inicialmente, la monetización a medio plazo podría explorarse mediante patrocinios de instituciones educativas, publicidad no intrusiva de material académico, o la futura implementación de un modelo *freemium* para tutorías privadas, lo cual amortizará la inversión inicial de 8.400 €.

Parte 4 Áreas de gestión del proyecto

4.1. Gestión de riesgos

Identificar y mitigar los riesgos es crucial para evitar desviaciones en la planificación o el presupuesto del proyecto MERN.

Riesgo	Probabilidad	Impacto	Plan de Contingencia
Retrasos en la integración de WebSockets	Media	Alto	Asignar más horas en las semanas 9-10; utilizar librerías simplificadas de Socket.io o reducir el alcance del chat a recarga asíncrona si el tiempo es crítico.
Brechas de seguridad en subida de archivos	Baja	Alto	Restringir por backend (Express) los tipos de archivos permitidos (solo PDF e imágenes), limitar el tamaño máximo

			a 10MB y utilizar almacenamiento seguro.
Baja adopción de usuarios al inicio	Alta	Medio	Crear grupos "semilla" con contenido pre-cargado (apuntes reales de asignaturas comunes) para que los primeros usuarios encuentren valor inmediato.
Sobrecarga de la base de datos (MongoDB)	Baja	Alto	Implementar paginación en el historial del chat y en la carga de recursos desde el primer día para no saturar las peticiones.

4.2. Gestión del equipo del proyecto (personal)

El proyecto cuenta con una estructura ágil orientada al desarrollo web moderno. Las funciones se distribuyen de la siguiente manera:

- **Jefe de Proyecto / Scrum Master:** Encargado de velar por el cumplimiento del cronograma, gestionar los riesgos y actuar como puente entre los requisitos funcionales y el equipo técnico.
- **Desarrollador Backend:** Responsable de la arquitectura del servidor (Node.js/Express), la seguridad de la API, la configuración de MongoDB y la lógica de WebSockets.
- **Desarrollador Frontend:** Encargado de plasmar el diseño UI/UX en código utilizando React y Tailwind CSS, asegurando la responsividad y el comportamiento SPA.
- **QA / Tester (Transversal):** Encargado de realizar pruebas de estrés, pruebas de los *endpoints* con Postman y validar la usabilidad de la interfaz.